

Desafio 2 — Post automático no TikTok com n8n e IA: guia completo

Oct 6, 2026 · @Paulo Arthur

1. Visão do produto

Esta automação é uma "produtora de vídeos" que trabalha sozinha. Ela escolhe um tema, escreve o roteiro com inteligência artificial, monta um vídeo vertical com fotos, narração, legendas e música, pede a sua aprovação pelo Telegram e publica no TikTok. **Foi testada de verdade:** em 06/10/2026 publicou vídeos na conta de teste pela API oficial do TikTok.

O problema

Estar presente nas redes exige constância, e cada vídeo curto consome de 1 a 2 horas de trabalho:

1. pensar no tema e no gancho;
2. escrever o roteiro;
3. gravar a narração;
4. buscar imagens;
5. montar legendas, transições e música no formato vertical;
6. escrever a legenda do post e as hashtags;
7. publicar.

Na prática, empresas postam pouco ou param.

A solução

O fluxo faz as sete etapas em cerca de 1 minuto por vídeo. A pessoa só decide se publica ou não.

Etapa	Quem faz
Escolher o tema	O agendamento diário, um formulário, o botão do n8n ou um pedido ao Claude (MCP)
Escrever o roteiro	IA gratuita (OpenRouter, com o Gemini de reserva) ou um banco de roteiros prontos
Gravar a narração	Voz neural em português, gratuita

Etapa	Quem faz
Buscar imagens	Banco de fotos livre (Pixabay), pelas palavras-chave que a IA escolheu
Montar o vídeo	Serviço próprio: fotos, emoji, texto, legendas palavra a palavra, música original e transições
Aprovar	Você, no Telegram, com os botões Publicar e Descartar
Publicar	API oficial do TikTok
Registrar	Histórico de cada publicação e de cada erro

Para quem é

- **Marketing e redes sociais:** frequência de posts sem produção manual.
- **Gestor:** pode pedir um vídeo conversando com o Claude e consultar o histórico.
- **Quem precisa de controle:** nada vai ao ar sem aprovação humana.

O que o case pedia e o que foi entregue

O case pedia	Entregue
Postagem automática para o TikTok	Roteiro, narração e vídeo gerados sozinhos
Publicação automática	Publicação real pela API oficial, com login OAuth e renovação do acesso
Ferramenta low-code, de preferência n8n	4 workflows no n8n
Usuários de teste	Sandbox do TikTok com conta de teste
Inovar	Aprovação humana, IA gratuita com reservas, visual rico, MCP, simulador, erros explicados
MCP (desejável)	Servidor MCP no próprio n8n
Sem custo não aprovado	Nada pago foi usado

Uma regra do TikTok explicada

Até o aplicativo passar pela auditoria do TikTok, ele só pode publicar em **contas privadas**, e os vídeos ficam visíveis só para o dono. É uma regra do TikTok, não um defeito, e vale para qualquer desenvolvedor. Por isso a conta de teste é privada.

2. A jornada do usuário

O usuário dispara um post de quatro formas, aprova no Telegram e acompanha tudo pelo n8n e pelo histórico.

2.1 Quatro formas de começar

Forma	Para quem	Como
Agendamento	Ninguém precisa agir	Todo dia às 11h, com o n8n ligado; o tema vem do rodízio do banco
Formulário	Quem quer escolher o tema	Abre o link do formulário, digita o tema (ou deixa vazio) e envia
Botão "Testar agora"	Quem demonstra ou configura	No n8n, <i>Execute workflow</i>
Conversa com o Claude (MCP)	Quem prefere linguagem natural	"Publica um vídeo sobre golpe do boleto"

2.2 O que o usuário vê

1. **No Telegram**, cerca de 40 a 60 segundos depois, chega o vídeo com a legenda do post e a pergunta "Publicar este vídeo no TikTok?", com os botões **Publicar** e **Descartar**.
2. **Publicar**: o fluxo envia o vídeo ao TikTok e espera a confirmação `PUBLISH_COMPLETE`. O vídeo aparece no perfil da conta de teste, com cadeado.
3. **Descartar**: nada é publicado, e o histórico registra `DESCARTADO`.
4. **Sem resposta em 12 horas**: a espera expira, e nada é publicado.

2.3 Onde ver os resultados

- **Telas do n8n**: cada etapa acende em verde. Ao clicar num nó, aparece o que ele produziu: o roteiro e a origem dele (`ia:` + modelo, ou `banco`), o vídeo, a resposta do TikTok.
- **Histórico**: `http://127.0.0.1:8765/log`, ou pelo Claude: "me mostra o histórico".
- **Execuções**: a aba *Executions* do n8n guarda todas, inclusive as com erro.

- **No TikTok:** perfil da conta de teste.

2.4 Quando algo dá errado

O fluxo para com uma mensagem que diz o que fazer, e o erro vai para o histórico com o link da execução.

Mensagem	O que fazer
"Conta TikTok não conectada"	Abrir o link de login e autorizar
"App sem auditoria só publica em CONTA PRIVADA"	Deixar a conta de teste privada
"Token inválido ou revogado"	Refazer o login
"Limite diário de posts do TikTok"	Tentar no dia seguinte

Se a IA falhar, o usuário nem percebe: outra IA, ou o banco de roteiros, assume, e o vídeo sai do mesmo jeito.

2.5 Configuração pelo usuário

Tudo se configura num arquivo `.env`, que fica só na máquina, e num comando: `python scripts\configurar.py`.

O que colocar no <code>.env</code>	Para quê	Obrigatório?
Chaves do app TikTok (sandbox) e URL de retorno	Publicar de verdade	Sim, para o modo real
Token do bot do Telegram	Aprovação humana	Não
Chave do OpenRouter (ou a do Desafio 1)	Roteiro por IA	Não
Chave do Gemini	Reserva da IA	Não
Chave do Pixabay ou Pexels	Fotos de fundo	Não

Sem nenhuma chave, o fluxo funciona no **modo simulador**: roteiro do banco, vídeo com fundo em degradê, publicação num TikTok simulado.

3. Fluxo de execução: o que é feito por IA e o que não é

Das 12 etapas de um post, só duas usam IA: escrever o roteiro e gerar a voz. Todo o resto é código determinístico (sempre o mesmo resultado para a mesma entrada), uma decisão humana ou uma chamada a uma API externa.

Só duas etapas usam IA; o resto é código, pessoa ou API

IA
Código determinístico
Pessoa
API externa

1. Gatilho agendamento, formulário, botão ou pedido ao Claude (MCP)	Pessoa / relógio
2. Config reúne tema, provedores de IA, aprovação, marca	Código (n8n)
3. Roteiro OpenRouter, depois Gemini, depois Gemini Lite; se todas falharem, banco	IA de texto
4. Normalizar roteiro limites de tamanho, emoji, hashtags, legenda do post; recusa roteiro inválido	Código (n8n)
5. Fotos de fundo busca no Pixabay pelas palavras-chave que a IA escolheu	API (Pixabay)
6. Narração voz neural pt-BR e o tempo de cada palavra (edge-tts)	IA de voz
7. Montagem do vídeo slides, legendas, trilha original e crossfade (Pillow, numpy, ffmpeg)	Código (Python)
8. Aprovação vídeo no Telegram com Publicar ou Descartar; o fluxo pausa até 12 h	Pessoa
9. Token válido lê o token e renova pelo refresh token se faltar menos de 5 min	Código (n8n)
10. Publicação consulta o criador, inicia, envia o arquivo, consulta o status	API (TikTok)
11. Conferir respostas traduz recusas da API em instrução; repete o status a cada 5 s	Código (n8n)
12. Registro histórico de cada post; falhas vão ao workflow de erros	Código (n8n)

fluxo de execução · 12 etapas, quem faz cada uma

A IA fica onde é preciso criatividade (roteiro) ou síntese de voz. O que tem regra clara (formato, limites, montagem, publicação, tratamento de erros) é código, porque código é previsível e testável.

3.1 Etapa por etapa

#	Etapa	Quem faz	Entrada → saída	Se falhar
1	Gatilho	Pessoa ou relógio	Tema opcional → início da execução	—
2	Config	Código (nó Set)	→ parâmetros da execução	—
3	Roteiro	IA de texto	Tema → JSON com gancho, cenas, emoji, palavras-chave de foto, CTA, hashtags	Próxima IA da cadeia; no fim, banco de roteiros
4	Normalizar	Código (nó Code)	JSON da IA → roteiro padronizado + legenda do post	Erro claro (roteiro sem gancho ou com menos de 2 cenas)
5	Fotos	API do Pixabay	Palavras-chave → foto vertical	Fundo em degradê
6	Narração	IA de voz (edge-tts)	Fala da cena → MP3 + tempo de cada palavra	Cena muda, tempos estimados
7	Montagem	Código (Python: Pillow, numpy, ffmpeg)	Cenas + áudios → vídeo 1080×1920	Erro na execução
8	Aprovação	Pessoa (Telegram)	Vídeo → Publicar ou Descartar	Expira em 12 h sem publicar
9	Token	Código (subworkflow)	Tokens salvos → token válido (renova se preciso)	"Conta não conectada"
10	Publicação	API do TikTok	Vídeo + legenda → <code>publish_id</code>	Erro traduzido em instrução
11	Conferir	Código	Respostas da API → segue, repete ou para	Para com motivo claro
12	Registro	Código	Resultado → histórico (CSV)	Workflow de erros registra

3.2 O que a IA decide e o que ela não decide

A IA decide	A IA não decide
O texto do gancho, das cenas, da narração e da chamada final	Se o vídeo será publicado (decisão humana)
O emoji de cada cena	O tamanho máximo dos textos e o número de cenas (código corta)
As palavras-chave das fotos	Qual foto entra (o banco de imagens devolve; o código escolhe pela semente do roteiro)
As hashtags	O formato final da legenda (código monta)
A voz (síntese neural)	Privacidade, envio, status e erros (código + API)

3.3 Tempos típicos

Etapas	Tempo
Roteiro por IA	5 a 15 s (pode crescer se um provedor falhar e a reserva assumir)
Fotos + narração + montagem (4 a 6 cenas)	20 a 40 s
Aprovação	o tempo da pessoa
Envio e processamento no TikTok	5 a 15 s

4. Glossário: os conceitos, da teoria ao projeto

Cada termo tem duas partes: **o que é** (definição geral) e **no projeto** (como aparece aqui). Conceitos de IA já explicados no documento do Desafio 1, como LLM, prompt, token, agente e tool use, aparecem aqui só quando têm uso específico.

4.1 Automação low-code e n8n

Low-code / no-code

- O que é: plataformas em que se constrói software montando blocos visuais, com pouco ou nenhum código. Aceleram automações e permitem que pessoas não programadoras entendam e alterem o fluxo.

- No projeto: o fluxo é desenhado no n8n. Código aparece só em pequenos trechos JavaScript dentro de nós *Code*, e num serviço auxiliar em Python para montar o vídeo.

n8n

- O que é: plataforma de automação de código aberto, alternativa a Zapier e Make. Pode ser instalada no próprio computador ou servidor, sem custo de licença. Tem centenas de integrações prontas e nós de IA.
- No projeto: versão 2.41.7, rodando localmente em `http://localhost:5678`. Os 4 workflows do desafio vivem nele.

Workflow (fluxo de trabalho)

- O que é: uma sequência de passos automatizados, desenhada como um diagrama de caixas ligadas por setas.
- No projeto: são 4 workflows: post automático (38 nós), autenticação (20 nós), tratamento de erros (2 nós) e servidor MCP (3 nós).

Nó (node)

- O que é: cada caixa do workflow. Faz uma coisa: receber um gatilho, chamar uma API, decidir um caminho, transformar dados. Os dados saem de um nó e entram no seguinte como *itens* em JSON.
- No projeto: os tipos usados são Manual Trigger, Schedule Trigger, Form Trigger, Execute Workflow Trigger, Set, If, Code, HTTP Request, Wait, Execute Workflow, Webhook, Respond to Webhook, Stop and Error, Error Trigger, Telegram, MCP Server Trigger, Call n8n Workflow Tool e HTTP Request Tool.

Gatilho (trigger)

- O que é: o nó que inicia o workflow: um horário, um formulário enviado, uma chamada externa ou um clique.
- No projeto: o fluxo principal tem 4 gatilhos que convergem para o mesmo nó *Config*.

Expressão (`{{ }}`)

- O que é: no n8n, um trecho entre `{{ }}` é calculado na hora, em JavaScript, a partir dos dados dos nós anteriores.
- No projeto: por exemplo, `{{ $('Renderizar vídeo').first().json.video_url }}` busca o link do vídeo produzido por outro nó, e `$runIndex` conta quantas vezes um nó já rodou.

Subworkflow

- O que é: um workflow chamado por outro, como uma função reutilizável.

- No projeto: o nó "Token TikTok válido" chama o workflow de autenticação, que devolve um token pronto para uso. O servidor MCP chama o fluxo principal da mesma forma.

Error workflow (fluxo de erros)

- O que é: um workflow que o n8n dispara automaticamente quando outro falha.
- No projeto: "TikTok · Tratamento de erros" registra no histórico o nó que falhou, a mensagem e o link da execução.

Publicar (ativar) um workflow

- O que é: no n8n 2.x, deixar um workflow "no ar", para que seus gatilhos (agendamento, webhooks, formulário, MCP) fiquem ouvindo.
- No projeto: os 4 workflows precisam estar publicados. Após ligar o n8n, o registro dos webhooks leva cerca de 20 segundos.

4.2 Comunicação entre sistemas

API e REST

- O que é: API é o "balcão" pelo qual um programa pede algo a outro. REST é o estilo mais comum de API web: cada recurso tem um endereço (URL) e se usa um *verbo* HTTP para agir sobre ele.
- No projeto: três APIs são usadas: a **TikTok Content Posting API** (publicar), a **Claude API** (roteiro) e a API do próprio serviço de mídia (vídeo, tokens, histórico).

HTTP e seus verbos

- O que é: o protocolo da web. **GET** busca, **POST** envia ou cria, **PUT** grava um conteúdo num endereço. Cada resposta traz um código: 200 (ok), 201 (criado), 307 (redirecionamento), 400 (pedido inválido), 401 (sem autorização), 404 (não encontrado).
- No projeto: o vídeo é enviado ao TikTok com **PUT**, e o TikTok responde 201. O login devolve 307 para levar o navegador ao TikTok.

Cabeçalho (header)

- O que é: informações extras que acompanham um pedido HTTP: quem você é, o tipo do conteúdo, o tamanho.
- No projeto: **Authorization: Bearer <token>** identifica a conta; **Content-Type: video/mp4** diz que é vídeo; **Content-Range: bytes 0-845747/845748** diz qual pedaço do arquivo está sendo enviado.

JSON

- O que é: formato de texto para dados estruturados (**{"status": "PUBLISH_COMPLETE"}**).

- No projeto: todos os dados que circulam entre os nós e as APIs são JSON. O roteiro também.

Webhook

- O que é: um endereço web que, ao receber uma chamada, dispara uma ação. É o contrário de perguntar o tempo todo ("tem novidade?"): o outro sistema avisa quando algo acontece.
- No projeto: `/webhook/tiktok/login` e `/webhook/tiktok/callback` são webhooks do n8n usados no login do TikTok.

Polling (consulta periódica)

- O que é: perguntar repetidamente, em intervalos, se algo terminou.
- No projeto: depois do envio, o fluxo espera 5 segundos e pergunta ao TikTok o status. Repete até "publicado", "falhou" ou 25 tentativas.

Localhost, porta e túnel HTTPS

- O que é: *localhost* é o próprio computador; a *porta* separa os serviços (5678, 8765). *HTTPS* é HTTP com criptografia. Um *túnel* (cloudflared, ngrok) cria um endereço público HTTPS que leva a um serviço local.
- No projeto: o n8n roda em `localhost:5678` e o serviço de mídia em `127.0.0.1:8765`. Para o login real, o TikTok exige um endereço HTTPS, então é preciso um túnel.

4.3 Autenticação e segurança

OAuth 2.0

- O que é: o padrão de "entrar com o Google/Facebook/TikTok". Permite que um aplicativo aja em nome de um usuário **sem receber a senha dele**. O usuário autoriza na página oficial; o aplicativo recebe um código e o troca por um token.
- No projeto: é o fluxo de autorização (*authorization code*) do TikTok, implementado no workflow de autenticação (seção 6).

Client key e client secret

- O que é: o "usuário e senha" do aplicativo (não da pessoa) junto ao provedor. O secret nunca deve ser exposto.
- No projeto: são fornecidos pelo TikTok for Developers e preenchidos no nó Config do workflow de autenticação.

Access token e refresh token

- O que é: o *access token* é um "crachá temporário" enviado em cada pedido à API. O *refresh token* serve para pegar um crachá novo sem o usuário autorizar de novo.

- No projeto: no TikTok, o access token dura 24 horas e o refresh token, 365 dias. O fluxo renova sozinho quando faltam menos de 5 minutos para o token vencer.

Escopo (scope)

- O que é: as permissões pedidas pelo aplicativo.
- No projeto: `user.info.basic` (dados básicos) e `video.publish` (publicar).

State e CSRF

- O que é: CSRF é um ataque em que um site malicioso faz o navegador da vítima disparar ações em outro site. No OAuth, o parâmetro `state` é um valor enviado na ida e conferido na volta, para garantir que a resposta veio de um login que o próprio aplicativo iniciou.
- No projeto: o callback só aceita a resposta se o `state` bater. Um state errado recebe a resposta HTTP 400 "Callback inválido" (testado).

Redirect URI

- O que é: o endereço para onde o provedor manda o usuário de volta após autorizar. Precisa estar cadastrado previamente.
- No projeto: `https://<túnel>/webhook/tiktok/callback`.

4.4 TikTok

TikTok for Developers e app

- O que é: o portal onde se registra um aplicativo para usar as APIs do TikTok.
- No projeto: o app precisa dos produtos *Login Kit* e *Content Posting API* (passo a passo em `docs/setup-tiktok.md`).

Sandbox e usuário de teste

- O que é: um ambiente de testes do provedor. Contas cadastradas como *target users* podem usar o app antes da aprovação.
- No projeto: é a forma de atender ao item do case "crie usuários de teste".

Content Posting API e Direct Post

- O que é: a API oficial do TikTok para enviar vídeos. O modo *Direct Post* publica direto no perfil. O modo *Upload* só manda para os rascunhos.
- No projeto: usa *Direct Post* com a sequência oficial: consultar o criador, iniciar a publicação, enviar o arquivo e consultar o status.

Auditoria do app e `SELF_ONLY`

- O que é: a revisão do TikTok que libera posts públicos. Sem ela, todo post é forçado a `SELF_ONLY` (privado).

- No projeto: o fluxo consulta as opções de privacidade e usa a preferida só se ela estiver disponível.

FILE_UPLOAD e chunk

- O que é: duas formas de entregar o vídeo ao TikTok. *FILE_UPLOAD* envia o arquivo; *PULL_FROM_URL* faz o TikTok baixá-lo de um site, mas exige domínio verificado. Arquivos grandes vão em pedaços (*chunks*) de 5 a 64 MB.
- No projeto: *FILE_UPLOAD* em 1 pedaço, porque os vídeos têm cerca de 1 MB.

publish_id

- O que é: o identificador que o TikTok dá a uma publicação em andamento, usado para consultar o status.
- No projeto: é guardado no histórico (ex.: `v_pub_mock_43b16659ffe3`).

is_aigc

- O que é: marcação de "conteúdo gerado por IA", exigida pelas políticas do TikTok.
- No projeto: enviada como `true` em todo post.

Rate limit

- O que é: o limite de chamadas que uma API aceita por intervalo de tempo.
- No projeto: o TikTok aceita 6 chamadas por minuto por token. O fluxo faz uma publicação por execução.

4.5 Vídeo e áudio

TTS (Text-to-Speech) e voz neural

- O que é: síntese de fala a partir de texto. Vozes *neurais* usam IA e soam naturais.
- No projeto: a biblioteca **edge-tts** usa as vozes neurais do Microsoft Edge, gratuitas. A voz é `pt-BR-AntonioNeural`, um pouco acelerada (+6%).

ffmpeg e ffprobe

- O que é: ferramentas de linha de comando, livres, para converter, cortar, juntar e analisar áudio e vídeo. São usadas em praticamente todo software de vídeo.
- No projeto: o *ffmpeg* transforma cada slide e sua narração num trecho de vídeo, com zoom e transição, e depois junta os trechos. O *ffprobe* mede a duração de cada narração.

Codec, H.264, AAC e MP4

- O que é: *codec* é o método de compactar vídeo ou áudio. H.264 é o codec de vídeo mais compatível; AAC, o de áudio. MP4 é o "envelope" que guarda os dois.

- No projeto: os vídeos saem em MP4 com H.264, AAC, 30 quadros por segundo e a opção `faststart` (começa a tocar antes de baixar tudo).

Formato 9:16 (1080 × 1920)

- O que é: o vídeo vertical, proporção de tela de celular, padrão do TikTok, Reels e Shorts.
- No projeto: cada quadro é desenhado em 1080 × 1920 pixels.

Pillow

- O que é: biblioteca Python para criar e editar imagens.
- No projeto: desenha os slides: fundo em degradê, brilho decorativo, texto centralizado com sombra, selo "VOCÊ SABIA?" e barra de progresso.

Zoom lento (efeito Ken Burns) e fade

- O que é: o zoom lento sobre uma imagem parada dá sensação de movimento. O *fade* é a transição de escurecer e clarear.
- No projeto: o filtro `zoompan` do `ffmpeg` aproxima até 6%, e `fade` faz entradas e saídas de 0,25 s.

4.6 IA, dados e engenharia

Structured output com JSON Schema

- O que é: obrigar o LLM a responder num formato exato, descrito por um esquema.
- No projeto: o roteiro tem que vir com `tema`, `gancho`, `cenas` (cada uma com `texto` e `narracao`, `emoji` e `imagem`), `emoji_gancho`, `imagem_gancho`, `cta` e `hashtags`.

Fallback

- O que é: o plano B automático quando o principal falha.
- No projeto: em cadeia. O roteiro tenta o OpenRouter; se falhar ou estourar a cota, o Gemini; depois o Gemini Lite; por fim, o banco de roteiros. Na montagem do vídeo, sem foto vira degradê e sem voz vira cena muda.

Human-in-the-loop e send-and-wait

- O que é: a máquina pausa e espera uma pessoa decidir. No n8n, o modo *send and wait* envia uma mensagem com botões e congela o fluxo até a resposta.
- No projeto: é a aprovação no Telegram, opcional.

MCP (Model Context Protocol)

- O que é: padrão aberto para assistentes de IA usarem sistemas externos (ver o glossário do Desafio 1).

- No projeto: o próprio n8n vira um **servidor MCP**, pelo nó *MCP Server Trigger*, em `http://localhost:5678/mcp/tiktok`. Ele expõe as ferramentas `criar_post_tiktok(tema)` e `historico_publicacoes()`. O transporte é HTTP.

Mock e simulador

- O que é: uma imitação de um serviço real, com o mesmo "contrato" (mesmos endereços, pedidos e respostas), para testar sem depender dele.
- No projeto: o serviço de mídia tem um simulador da API do TikTok em `/mock-tiktok`. Ele confere o tamanho do arquivo e o `Content-Range` e recusa privacidade diferente de `SELF_ONLY`, como o TikTok real faria com um app não auditado.

Fluxo como código

- O que é: manter a definição de um fluxo visual também como código-fonte, que gera o arquivo importável. Facilita revisar mudanças e versionar.
- No projeto: `workflows/src/build.py` gera os 4 arquivos JSON que o n8n importa.

Node.js

- O que é: ambiente que roda JavaScript fora do navegador. O n8n é feito nele.
- No projeto: o n8n 2.41 exige Node 24. Foi instalado em modo portátil, em `C:\Users\paulo\tools\node24`, sem alterar o Node 20 do sistema.

Task runner

- O que é: no n8n 2.x, processos separados que executam o código dos nós *Code*, isolados do n8n principal, por segurança.
- No projeto: os trechos JavaScript (banco de roteiros, normalização, preparação do post) rodam no task runner de JavaScript.

4.7 Termos acrescentados na segunda versão

Gemini e Google AI Studio

- O que é: família de modelos de IA do Google. O AI Studio dá uma chave gratuita, com cota diária por modelo, e um endereço compatível com o padrão da OpenAI.
- No projeto: é a reserva da IA do roteiro. O `configurar.py` mede qual modelo Flash responde mais rápido e usa esse; o Flash-Lite fica como segunda tentativa.

Cota (HTTP 429) e sobrecarga (HTTP 503)

- O que é: 429 significa "limite de uso atingido" (por minuto ou por dia); 503 significa "serviço temporariamente sobrecarregado".
- No projeto: os dois aconteceram nos testes, com o OpenRouter sem cota diária e o Gemini mais novo sobrecarregado. A cadeia de reservas existe por isso.

Banco de imagens com licença livre (Pixabay, Pexels)

- O que é: sites de fotos que permitem uso gratuito, inclusive comercial, sem exigir crédito ao autor, e que têm API.
- No projeto: fotos de fundo das cenas, buscadas pelas palavras-chave em inglês que a IA escolheu. O Pexels suspendeu novas chaves em 10/2026, então o projeto usa o Pixabay.

User-Agent e Cloudflare

- O que é: o User-Agent identifica o programa que faz o pedido. O Cloudflare é um serviço de proteção que pode bloquear programas que não se parecem com navegadores.
- No projeto: o Pixabay recusava o cliente Python padrão ("error code: 1010"). Os pedidos passaram a se identificar como navegador.

Legenda ASS (Advanced SubStation Alpha)

- O que é: formato de legenda com estilos, cores e tempos precisos, que o ffmpeg "queima" no vídeo.
- No projeto: legendas palavra a palavra. Grupos de até 3 palavras, quebrando na pontuação, com a palavra falada em amarelo e sem sobreposição.

Crossfade e sidechain

- O que é: crossfade é a transição em que uma cena se dissolve na outra. Sidechain é a técnica de áudio que abaixa a música automaticamente quando há fala.
- No projeto: filtros `xfade` e `acrossfade` (0,35 s) entre cenas; `sidechaincompress` para a trilha.

Trilha procedural

- O que é: música gerada por código, a partir de regras de harmonia, em vez de gravada.
- No projeto: `numpy` compõe uma trilha ambiente original para cada vídeo (progressão vi-IV-I-V, 96 BPM, tom variando por vídeo). Sem risco de direitos autorais.

Conta privada (regra de apps sem auditoria)

- O que é: o TikTok só deixa um app ainda não auditado publicar em contas configuradas como privadas.
- No projeto: descoberto na primeira tentativa real (`unaudited_client_can_only_post_to_private_accounts`). Agora o fluxo explica o erro, e o simulador reproduz a regra.

Telegram send-and-wait

- O que é: modo do nó do Telegram no n8n que envia uma mensagem com botões e pausa o fluxo até a resposta.
- No projeto: a aprovação humana. O botão abre uma página do n8n. Para funcionar no celular, o n8n precisa de endereço público (túnel).

5. Arquitetura técnica

O n8n é o maestro: decide a ordem dos passos e chama, por HTTP, cada serviço especializado. A IA escreve, o serviço de mídia monta o vídeo, a pessoa aprova no Telegram e o TikTok publica.

O n8n orchestra; IA, mídia, Telegram e TikTok são chamados por HTTP



peças da solução · 4 gatilhos, 1 orquestrador, 7 serviços

A seta tracejada indica que o workflow de erros não está no caminho normal: o n8n o dispara quando algo falha. O serviço de mídia também guarda os tokens e o histórico, que os outros workflows leem e gravam.

5.1 As peças

Peça	Tecnologia	Responsabilidade	Usa IA?
Workflow principal	n8n	Orquestrar tudo, normalizar o roteiro, conferir respostas	Não
Cadeia de IA	OpenRouter, Gemini (HTTP)	Escrever o roteiro	Sim
Banco de roteiros	n8n (nó Code)	Plano B sem IA: 5 roteiros prontos	Não
Serviço de mídia	Python: Pillow, numpy, ffmpeg	Montar o vídeo, guardar tokens e histórico, simular o TikTok	Não
Voz	edge-tts (serviço de voz neural)	Narração e tempos das palavras	Sim (síntese de voz)
Fotos	API do Pixabay (ou Pexels)	Foto vertical por cena	Não
Autenticação	n8n	Login OAuth no TikTok e token sempre válido	Não
Telegram	Bot do Telegram	Aprovação humana	Não (pessoa)
TikTok Content Posting API	TikTok	Publicar	Não
Servidor MCP	n8n	Expor o fluxo como ferramenta para o Claude	Não (o Claude do usuário é o cliente)
Tratamento de erros	n8n	Registrar falhas	Não

5.2 Decisões de projeto

Decisão	Motivo
Serviço de mídia por HTTP, não <i>Execute Command</i>	No n8n 2.x esse nó vem desligado por segurança; por HTTP o desenho funciona também com n8n em Docker ou na nuvem

Decisão	Motivo
Cadeia de IA com reservas e banco no fim	Modelos gratuitos têm cota pequena e picos de sobrecarga; o post não pode depender de um só
Saída da IA sempre normalizada por código	Mesmo com schema, a IA pode exagerar; o código garante limites e formato
Aprovação humana antes de publicar	Conteúdo gerado por IA vai para uma conta pública da marca: alguém precisa validar
Simulador da API com o mesmo contrato	Permite testar e avaliar sem conta e sem esperar a aprovação do TikTok
Fluxo como código (<code>build.py</code>) + configuração local fora do git	Revisão em PR, ids estáveis e nenhum segredo no repositório
Uma única chave entre simulador e real (<code>TIKTOK_MODAL</code>)	O workflow principal nunca sabe com qual TikTok está falando

5.3 Estrutura de pastas

```
desafio2-tiktok-n8n/
├─ workflows/
│   ├─ 01_tiktok_post_automatico.json    workflow principal (38 nós)
│   ├─ 02_tiktok_autenticacao.json       OAuth + token (20 nós)
│   ├─ 03_tiktok_erros.json               tratamento de erros (2 nós)
│   ├─ 04_tiktok_mcp_server.json          servidor MCP (3 nós)
│   └─ src/
│       ├─ build.py                       gera os JSONs (genéricos ou --local)
│       ├─ promptroteiro.md               instruções do roteirista (IA)
│       └─ bancoroteiros.js               5 roteiros prontos (plano B)
├─ media_service/
│   ├─ server.py                           API HTTP + simulador do TikTok
│   └─ render.py                           montagem do vídeo
├─ scripts/
│   └─ configurar.py                       .env -> credenciais no n8n -> aplica
workflows
│   ├─ aplicar_local.py                    gera, importa e ativa
│   ├─ n8n_api.py                          cliente da API interna do n8n
│   └─ testar_mcp.py                       cliente MCP de teste
├─ docs/setup-tiktok.md                   passo a passo do modo real
├─ docs/app-tiktok/
└─ privacidade
```

└─ assets/	vídeos e capas de exemplo
└─ .env.example	modelo de configuração
└─ iniciar.ps1 / importar.ps1	
└─ README.md	

6. O workflow principal, nó a nó

"TikTok · Post automático com IA" tem 38 nós em sete blocos. A coluna "IA?" mostra onde há inteligência artificial.

6.1 Gatilhos e configuração

Nó	Tipo	O que faz	IA?
Testar agora	Manual Trigger	Dispara pelo botão <i>Execute workflow</i>	Não
Agendado (diário 11h)	Schedule Trigger	Dispara todo dia às 11h (fuso de São Paulo)	Não
Formulário (tema manual)	Form Trigger	Página web com o campo "tema"	Não
Chamado via MCP / outro workflow	Execute Workflow Trigger	Recebe o <code>tema</code> do servidor MCP	Não
Config	Set	Reúne <code>tema</code> , <code>usar_ia</code> , <code>provedor_ia</code> , modelos (OpenRouter, Gemini, Gemini Lite, Claude), aprovação, chat do Telegram, marca	Não
Usar IA?	If	Com <code>usar_ia</code> ligado, vai para a IA; senão, para o banco	Não
Provedor: Claude?	If	<code>anthropic</code> vai para o Claude; o padrão vai para o OpenRouter	Não

6.2 Roteiro: a cadeia de IA

Cada nó de IA tem uma **saída de erro** (`onError: continueErrorOutput`): quando falha, em vez de parar o fluxo, passa o bastão para o próximo.

Nó	Tipo	O que faz	IA?	Se falhar, vai para
OpenRouter: gerar roteiro	HTTP Request	POST openrouter.ai/api/v1/chat/completions com o prompt e o JSON Schema do roteiro; 2 tentativas	Sim	Gemini
Ler resposta (OpenRouter)	Code	Extraí e converte o JSON da resposta; anota a origem <code>ia: <modelo></code>	Não	Gemini
Gemini: gerar roteiro (reserva)	HTTP Request	Endpoint do Gemini compatível com OpenAI; modelo Flash escolhido pela latência medida	Sim	Gemini Lite
Ler resposta (Gemini)	Code	Igual ao anterior	Não	Gemini Lite
Gemini Lite: segunda tentativa	HTTP Request	Modelo Flash-Lite, o mais disponível em picos	Sim	Banco
Ler resposta (Gemini Lite)	Code	Igual ao anterior	Não	Banco
Claude: gerar roteiro	HTTP Request	Alternativa paga (<code>provedor_ia = anthropic</code>), com <i>structured output</i>	Sim	Banco
Ler resposta da IA	Code	Confere recusa ou resposta cortada; extrai o JSON	Não	Banco
Banco de roteiros	Code	5 roteiros prontos, escolhidos pelo tema ou em rodízio pelo dia	Não	—
Normalizar roteiro	Code	Corta textos, limita a 5 cenas e 6 hashtags, garante um emoji por cena, monta a legenda do post (até 2.200 caracteres); recusa roteiro sem gancho ou com menos de 2 cenas	Não	Erro

6.3 Vídeo e aprovação

Nó	Tipo	O que faz	IA?
Renderizar vídeo	HTTP Request	<code>POST /render</code> no serviço de mídia; recebe link, tamanho, duração e quantas cenas tiveram foto e voz	Voz neural dentro do serviço
Exigir aprovação?	If	Liga ou desliga o bloco de aprovação	Não
Baixar prévia	HTTP Request	Baixa o vídeo como arquivo	Não
Telegram: enviar prévia	Telegram	Envia o vídeo com a legenda ao chat configurado	Não
Telegram: aprovar?	Telegram (send and wait)	Botões Publicar e Descartar; pausa até 12 h	Não (pessoa)
Aprovado?	If	Publicar segue; Descartar vai para o registro	Não
Registrar descarte	HTTP Request	Grava <code>DESCARTADO</code> no histórico	Não

Nos workflows do repositório, os dois nós do Telegram vêm desligados, porque o n8n não ativa nós sem credencial. O `configurar.py` os liga automaticamente quando há token do bot.

6.4 Publicação no TikTok

Nó	Tipo	O que faz
Token TikTok válido	Execute Workflow	Chama o subworkflow de autenticação; recebe <code>access_token</code> e <code>api_base</code> (real ou simulador)
Consultar criador	HTTP Request	<code>POST /v2/post/publish/creator_info/query/</code> , exigido antes de postar
Preparar post	Code	Escolhe a privacidade permitida e monta o pedido: legenda, <code>SELF_ONLY</code> , <code>is_aigc: true</code> , <code>FILE_UPLOAD</code> em 1 pedaço

Nó	Tipo	O que faz
Iniciar publicação	HTTP Request	POST /v2/post/publish/video/init/ , sem parar em erro HTTP, para que o próximo nó leia o motivo
Conferir início	Code	Se o TikTok recusou, traduz o código em instrução (ex.: "deixe a conta privada")
Baixar vídeo	HTTP Request	Pega o arquivo do serviço de mídia
Enviar vídeo (PUT)	HTTP Request	PUT no upload_url com Content-Type: video/mp4 e Content-Range
Aguardar processamento	Wait	5 segundos
Consultar status	HTTP Request	POST /v2/post/publish/status/fetch/
Publicado?	If	PUBLISH_COMPLETE segue para o registro
Falhou ou esgotou?	If	FAILED ou 25 tentativas: erro; senão, volta a aguardar
Erro na publicação	Stop and Error	Para com mensagem clara e dispara o workflow de erros

Nenhum nó deste bloco usa IA.

6.5 Registro e resultado

Nó	Tipo	O que faz
Registrar no log	HTTP Request	Grava tema, título, publish_id , status, privacidade, link do vídeo e modo (api-real ou simulador)
Resultado	Set	Resumo final; é o que o Claude recebe quando o fluxo é chamado pelo MCP

6.6 Configurações do workflow

- errorWorkflow : aponta para "TikTok · Tratamento de erros".
- callerPolicy: any : permite que o servidor MCP chame este workflow.
- executionOrder: v1 : ordem de execução atual do n8n.

7. Autenticação, tratamento de erros e servidor MCP

Os três workflows de apoio cuidam do acesso à conta, das falhas e do acesso pelo Claude. Nenhum deles usa IA.

7.1 "TikTok · Autenticação (OAuth + token)": 20 nós, 3 entradas

As três entradas compartilham a mesma configuração: `client_key` e `client_secret` do sandbox, `redirect_uri`, escopos, `api_base` (real ou simulador), endereço do serviço de mídia e `state`. Esses valores vêm do `.env`, pelo `configurar.py`, e nunca vão para o git.

Entrada 1: login (GET /webhook/tiktok/login)

1. Monta o endereço oficial de autorização com `client_key`, escopos `user.info.basic,video.publish`, `redirect_uri` e `state`.
2. Redireciona o navegador para a tela do TikTok (HTTP 307).

Entrada 2: retorno (GET /webhook/tiktok/callback)

1. Confere o `state` (proteção contra CSRF) e a presença do `code`; senão, HTTP 400.
2. Troca o `code` por `access_token` (24 h) e `refresh_token` (365 dias) em POST `/v2/oauth/token/`.
3. Guarda os tokens e o `api_base` no serviço de mídia (`tokens.json`, fora do git).
4. Mostra "Conta TikTok conectada".

Entrada 3: token para o post (chamada pelo workflow principal)

1. Lê os tokens. Sem tokens, para com "Conta TikTok não conectada: abra .../webhook/tiktok/login".
2. Se faltam mais de 5 minutos para vencer, devolve o token atual.
3. Se não, renova com o `refresh_token`, guarda e devolve o novo.

O TikTok exige que a Redirect URI seja HTTPS. Por isso o login usa um túnel gratuito (Cloudflare) até o n8n local. Depois do login, o túnel só é necessário para aprovar pelo celular.

7.2 "TikTok · Tratamento de erros": 2 nós

1. **Error Trigger:** o n8n o dispara sempre que o workflow principal falha.
2. **Registrar erro:** grava no histórico o status ERRO, o nó que falhou, a mensagem e o link da execução.

Exemplo real, da primeira tentativa de post com a conta ainda pública: "Iniciar publicação: Forbidden". Isso levou à criação do nó "Conferir início". Hoje a mensagem seria: "TikTok recusou a publicação (`unaudited_client_can_only_post_to_private_accounts`). App

sem auditoria só publica em CONTA PRIVADA...".

7.3 "TikTok · Servidor MCP": 3 nós

Nó	O que faz
MCP Server Trigger	Publica um servidor MCP em <code>http://localhost:5678/mcp/tiktok</code>
<code>criar_post_tiktok</code>	Chama o workflow principal com o parâmetro <code>tema</code> , que o Claude preenche (expressão <code>\$fromAI</code>)
<code>historico_publicacoes</code>	Lê o histórico de posts

Como usar: `claude mcp add --transport http tiktok-n8n`
`http://localhost:5678/mcp/tiktok`. Depois: "publica um vídeo sobre golpe do boleto e me mostra o histórico".

Com a aprovação ligada, a ferramenta responde que o fluxo está pausado aguardando decisão humana. O post segue sozinho quando alguém toca em Publicar no Telegram. Foi assim que os posts reais de validação foram disparados.

Quem é a IA aqui? O servidor MCP não usa IA. Quem usa é o cliente, o Claude do usuário, que interpreta o pedido em linguagem natural e decide chamar a ferramenta com um tema.

8. O serviço de mídia: como o vídeo é montado e o simulador

O serviço de mídia é um programa Python em `http://127.0.0.1:8765`. Ele transforma o roteiro num vídeo de 20 a 40 segundos em cerca de 30 segundos. Também guarda tokens e histórico e simula a API do TikTok. A única IA aqui é a síntese de voz.

8.1 Rotas

Rota	O que faz
<code>GET /health</code>	Indica se o serviço está no ar
<code>POST /render</code>	Roteiro → vídeo; devolve link, tamanho, duração, cenas com voz e com foto
<code>GET /videos/<id>/<arquivo></code>	Entrega o <code>video.mp4</code> ou a <code>capa.png</code>

Rota	O que faz
GET/POST /tokens	Lê ou grava os tokens OAuth (<code>tokens.json</code> , fora do git)
GET/POST /log	Lê ou acrescenta ao histórico (<code>output/publicacoes.csv</code>)
/mock-tiktok/...	Simulador da API do TikTok

8.2 A montagem, passo a passo (`media_service/render.py`)

1. Cenas. O gancho vira a primeira cena, com o selo "VOCÊ SABIA?"; depois as cenas do roteiro; por fim a chamada final, com "Siga para mais automações".

2. Fundo de cada cena. Busca no Pixabay (ou Pexels) uma foto vertical pelas palavras-chave em inglês que a IA escolheu. A escolha entre os resultados é determinística (semente derivada do roteiro), e as fotos ficam em cache. A foto é recortada em 9:16, escurecida no topo e na base para legibilidade e recebe um leve tom da marca. Sem chave ou sem rede, entra um degradê.

3. Desenho do slide (Pillow).

- emoji da cena, colorido, num selo circular sobre a foto;
- texto principal num painel translúcido com sombra, com tamanho de fonte ajustado ao comprimento;
- marca no topo e barra de progresso na base.

4. Narração (IA de voz). O `edge-tts` gera a fala com a voz neural `pt-BR-AntonioNeural` e devolve o instante e a duração de cada palavra. A cena dura o mesmo que a fala, mais 0,45 s.

5. Trecho de vídeo por cena (ffmpeg). Imagem + áudio, com zoom lento de até 8% (efeito Ken Burns).

6. Legendas dinâmicas. A partir dos tempos das palavras, é gerado um arquivo ASS:

- grupos de até 3 palavras, quebrando na pontuação (recuperada do texto original);
- a palavra que está sendo falada aparece em amarelo;
- cada legenda termina quando a próxima começa, sem sobreposição.

7. Trilha original. O `numpy` compõe uma música ambiente: pad de acordes na progressão vi-IV-I-V e arpejo suave, a 96 BPM, num tom que varia por vídeo. Sem direitos autorais.

8. Junção final (um comando ffmpeg).

- crossfade de 0,35 s entre as cenas, no vídeo (`xfade`) e no áudio (`acrossfade`);

- legendas "queimadas" no vídeo;
- trilha a 32% do volume, abaixando sozinha quando há fala (`sidechaincompress`);
- fade de entrada e saída; H.264 + AAC; `faststart` .

Exemplo real (post de 06/10)	Valor
Resolução	1080 × 1920
Cenas	6, todas com voz e foto
Tamanho	cerca de 3,6 MB
Roteiro	escrito pelo <code>gemini-3.5-flash</code>

8.3 Simulador da API do TikTok (`/mock-tiktok`)

Implementa os mesmos endereços e regras da API real, para testar sem conta:

Endereço	Comportamento
<code>POST /v2/oauth/token/</code>	Devolve tokens de teste
<code>POST /v2/post/publish/creator_info/query/</code>	Exige <code>Bearer</code> ; devolve as três opções de privacidade, como a API real
<code>POST /v2/post/publish/video/init/</code>	Exige <code>FILE_UPLOAD</code> em 1 pedaço e <code>SELF_ONLY</code> ; com <code>MOCK_CONTA_PRIVADA=0</code> , reproduz a recusa de conta pública
<code>PUT /upload/<id></code>	Confere tamanho e <code>Content-Range</code> ; responde 201
<code>POST /v2/post/publish/status/fetch/</code>	<code>PROCESSING_UPLOAD</code> e, cerca de 3 s depois, <code>PUBLISH_COMPLETE</code>

A troca simulador ↔ real é um campo no `.env` (`TIKTOK_MODO`).

9. Como a IA é usada

A IA de texto tem um papel só: escrever o roteiro. Ela recebe um tema e devolve um JSON. O código valida, corrige limites e decide o resto.

9.1 A cadeia de provedores

Ordem	Provedor	Modelo	Cota gratuita	Quando entra
1	OpenRouter	nvidia/nemotron-3-super-120b-a12b:free	50 chamadas/dia por conta	Sempre que usar_ia está ligado
2	Gemini	Flash mais rápido no momento da configuração (ex.: gemini-3.5-flash)	~20 chamadas/dia por modelo	OpenRouter falhou, recusou ou estourou a cota
3	Gemini Lite	gemini-3.5-flash-lite	~20 chamadas/dia	O Flash falhou (ex.: 503 de sobrecarga)
4	Banco de roteiros	— (sem IA)	ilimitado	Todas as IAs falharam, ou usar_ia desligado
alt.	Claude	claude-opus-5-5	pago	Se provedor_ia = anthropic

Em 06/10/2026, com o OpenRouter sem cota, o Gemini escreveu o roteiro do post "previsão de fluxo de caixa". O resultado foi publicado de verdade.

9.2 O prompt (workflows/src/prompt_roteiro.md)

O prompt de sistema coloca a IA no papel de roteirista de vídeos de 20 a 40 segundos sobre automação e IA para empresas, em português do Brasil.

Campo	Regra
gancho	Até 12 palavras, para prender nos 2 primeiros segundos
emoji_gancho , imagem_gancho	1 emoji; 2 a 4 palavras-chave em inglês para a foto
cenas (3 ou 4)	texto na tela (até 8 palavras), narracao (até 25 palavras), emoji , imagem
cta	Chamada curta: seguir, comentar uma palavra, salvar
hashtags	4 a 6, em minúsculas

Campo	Regra
Conteúdo	Uma ideia prática e verdadeira; sem promessas exageradas; sem marcas ou pessoas reais

9.3 Como o formato é garantido

1. **No pedido:** OpenRouter e Claude recebem um JSON Schema (*structured output*). O Gemini recebe o formato no prompt e o modo `json_object`.
2. **Na leitura:** os nós "Ler resposta" extraem o JSON mesmo se vier entre crases. Recusa ou resposta cortada viram erro, e a cadeia segue.
3. **Na normalização:** o código corta textos longos, limita cenas e hashtags, garante um emoji por cena e recusa roteiros sem gancho ou com menos de 2 cenas.

9.4 O banco de roteiros (sem IA)

Cinco roteiros prontos: nota fiscal, golpe do boleto, n8n, chão de fábrica e planilha. Cada um já tem emoji e palavras-chave de foto por cena. A escolha é pelo tema digitado (ignorando acentos) ou em rodízio pelo dia do ano.

9.5 A IA de voz

A narração usa o edge-tts, que acessa as vozes neurais do Microsoft Edge. É gratuito e sem chave. Além do áudio, devolve o tempo de cada palavra, base das legendas dinâmicas. Sem rede, a cena fica muda, com tempos estimados.

9.6 Custos e cotas

Item	Custo
OpenRouter (modelo gratuito)	R\$ 0, 50 roteiros/dia
Gemini e Gemini Lite (plano gratuito)	R\$ 0, ~20 roteiros/dia por modelo
Voz (edge-tts)	R\$ 0
Claude (opcional)	~US\$ 0,03 por roteiro

Para um post por dia, as cotas gratuitas sobram. Para dezenas de testes no mesmo dia, a cadeia de reservas e o banco garantem que o fluxo nunca para.

10. Como foi construído e como foi testado

10.1 Workflows gerados por código

Os workflows não foram desenhados à mão. O `workflows/src/build.py` gera os quatro JSON, com cada nó, posição e ligação. Isso traz três vantagens:

- **Revisão:** mudanças aparecem no git como diff legível.
- **Reprodução:** qualquer pessoa importa os JSON e tem o mesmo fluxo.
- **Dois sabores:** `python build.py` gera a versão genérica, com placeholders, que vai para o repositório. `python build.py --local` gera a versão com IDs de credencial e endereços da máquina, a partir de `local.json`, fora do git. O `scripts/aplicar_local.py` envia essa versão ao n8n pela API.

O `scripts/configurar.py` faz o resto da preparação. Ele testa cada chave do `.env`, mede a latência dos modelos Gemini e escolhe o mais rápido, descobre o `chat_id` do Telegram e cria as credenciais no n8n, sem imprimir nenhum segredo.

10.2 Testes no simulador

O fluxo completo foi rodado várias vezes contra o simulador: login, token, `creator_info`, envio, status e histórico. Temas testados: chão de fábrica, planilha, nota fiscal, conciliação bancária e golpe do boleto. Todos terminaram em `PUBLISH_COMPLETE`. Também foram testados:

- **Descartar no Telegram:** nada é publicado e o histórico registra `DESCARTADO`.
- **Conta pública no simulador** (`MOCK_CONTA_PRIVADA=0`): a recusa vira mensagem clara.
- **Servidor MCP:** `scripts/testar_mcp.py` lista as ferramentas, cria um post e lê o histórico.

10.3 Testes com o TikTok de verdade

Cinco vídeos foram publicados de verdade na conta de teste do sandbox (privada), em 06/10 e 08/10/2026, todos com aprovação pelo Telegram:

Hora	Tema	Roteiro por	Visual	publish_id
17:52	golpe do boleto	banco de roteiros	primeira versão (degradê)	v_pub_file~v2-1.7693659920302147605
18:01	Automação de notas fiscais	banco de roteiros	primeira versão	v_pub_file~v2-1.7693662703646312454

Hora	Tema	Roteiro por	Visual	publish_id
18:45	conciliação bancária	banco de roteiros	novo (fotos, emoji, legendas, trilha)	v_pub_file~v2-1.7693673373330737172
18:49	Previsão de fluxo de caixa automatizada	gemini-3.5-flash (OpenRouter sem cota)	novo	v_pub_file~v2-1.7693674964672235540
08/10 15:49	Automação de Contas a Pagar com IA	gemini-3.5-flash	novo	v_pub_file~v2-1.7694370358614394900

Todos chegaram a PUBLISH_COMPLETE e aparecem com cadeado no perfil da conta de teste.

10.4 Problemas encontrados e como foram resolvidos

Problema	Causa	Solução
Primeira publicação real: "Forbidden"	App sem auditoria só publica em conta privada	Conta de teste privada; nó "Conferir início" traduz o código de erro
Botões do Telegram não abriam no celular	Link apontava para localhost	n8n iniciado com o túnel como N8N_WEBHOOK_URL
OpenRouter: 429	Cota de 50 chamadas/dia esgotada nos testes	Reserva Gemini na cadeia
Gemini: 503	Sobrecarga do modelo mais procurado	Escolha do modelo por latência e segunda tentativa no Flash-Lite
Pexels sem novas chaves	Emissão suspensa pelo site	Pixabay como alternativa (o Pexels continua aceito)
Pixabay: "chave recusada"	Bloqueio do Cloudflare para clientes sem User-Agent (erro 1010)	User-Agent de navegador
Webhooks 404 logo após subir	n8n ainda registrando os gatilhos	demo.ps1 espera ~20 s depois do healthz

Problema	Causa	Solução
Gemini Lite chamava o modelo errado	Marcador do campo não aplicado no corpo	Corrigido no gerador e testado
n8n não instalava	Exige Node 24	Node 24 portátil em C:\Users\paulo\tools

10.5 Conferências finais

- **Gerador:** `build.py` (genérico e local) roda sem erro; os JSON genéricos não têm IDs, chaves nem endereços da máquina.
- **Workflows:** os quatro estão ativos no n8n; o principal tem 38 nós.
- **Segredos:** nenhum segredo no histórico do git. `.env`, `tokens.json`, `local.json`, `workflows/local/` e `output/` estão no `.gitignore`.

11. Operação, custos, limitações e próximos passos

11.1 Rotina de operação

Quando	O que fazer
Todo dia	Nada: às 10h o fluxo gera o vídeo e manda para o Telegram. Basta tocar em Publicar ou Descartar
Post extra	Formulário, botão "Execute workflow" ou pedido ao Claude pelo MCP
Algo falhou	Ver o histórico (<code>/log</code>) e a aba Executions; a mensagem diz o motivo e a ação
A cada 365 dias	Refazer o login (<code>/webhook/tiktok/login</code>); o token de 24 h se renova sozinho
Ao reiniciar o PC	<code>.\demo.ps1 -Somente d2</code> ; se o túnel mudar de endereço, reiniciar o n8n com o novo <code>N8N_WEBHOOK_URL</code>

11.2 Custos

Item	Hoje (case)	Em produção
n8n	R\$ 0 (local)	R\$ 0 auto-hospedado num servidor de ~R\$ 30/mês, ou n8n Cloud a partir de ~€ 24/mês
IA de roteiro	R\$ 0 (OpenRouter + Gemini gratuitos)	~US\$ 1/mês com Claude para um post por dia
Voz, trilha, montagem	R\$ 0	R\$ 0
Fotos	R\$ 0 (Pixabay)	R\$ 0
TikTok API	R\$ 0	R\$ 0 (exige auditoria do app)
Túnel	R\$ 0 (Cloudflare temporário)	R\$ 0 com domínio próprio no Cloudflare Tunnel

11.3 Limitações conhecidas

- **Visibilidade:** enquanto o app não passa pela auditoria do TikTok, só publica como privado (`SELF_ONLY`) em conta privada. É regra da plataforma, não do projeto.
- **Cotas gratuitas:** cerca de 50 + 40 roteiros por dia somando OpenRouter e Gemini. Além disso, entra o banco de roteiros, sem IA.
- **Dados no plano gratuito:** os provedores gratuitos podem usar os textos enviados. Os temas são públicos, então o risco é baixo.
- **Execução local:** o computador precisa estar ligado às 10h, e o túnel temporário muda de endereço a cada reinício.
- **Voz:** o edge-tts é um serviço não oficial. Se sair do ar, o vídeo sai sem narração, mas com legendas estimadas.
- **Fotos:** escolhidas por palavras-chave; nem sempre ilustram a cena com precisão.

11.4 Próximos passos para produção

1. **Auditoria do app** no TikTok, para publicar em público em contas reais.
2. **Servidor sempre ligado** com n8n e serviço de mídia em Docker, e túnel com domínio fixo.
3. **Calendário editorial:** temas vindos de uma planilha ou do Notion, em vez do rodízio.
4. **Métricas:** ler visualizações e curtidas pela API e realimentar a escolha de temas.
5. **Mais redes:** o mesmo vídeo em Reels e Shorts, com um nó de publicação por rede.

6. Integração com o Desafio 1: temas sugeridos a partir dos casos reais de fraude e divergência que o agente de contas a pagar encontra, sem dados sensíveis.